

Product Requirements Document

SaaS Personal Budgeting (Multi-Tenant MVP) & Panduan Setup Teknis

1. Problem Statement & Value Proposition

Problem Statement

Banyak pengguna kesulitan mengelola keuangan pribadi karena aplikasi *budgeting* yang ada di pasaran seringkali terlalu kompleks, mahal, atau tidak memberikan jaminan privasi data yang transparan. Mengingat data finansial sangat sensitif, risiko kebocoran data (*data breach*) antar pengguna menjadi kekhawatiran utama yang sering diabaikan oleh platform standar.

Value Proposition

Aplikasi ini menyediakan platform **SaaS Personal Budgeting** yang ringan, cepat, dan fokus pada kemudahan penggunaan. Melalui pendekatan **Multi-Tenant** dengan isolasi basis data di level baris (Row Level Security), setiap pengguna (tenant) mendapatkan *workspace* yang sepenuhnya terenkapsulasi. Hal ini menjamin bahwa data finansial pengguna 100% aman, privat, dan tidak mungkin terakses oleh tenant lain, sekaligus memungkinkan sistem berskala besar dari satu *codebase* dan *database* yang efisien.

2. Minimum Viable Product (MVP) Features

A. Autentikasi & Registrasi Tenant

- **Sign-Up / Log-In:** Pengguna mendaftar menggunakan Email dan Password, di-handle dengan aman oleh infrastruktur Auth.

- **Tenant Isolation:** Saat registrasi berhasil, sistem otomatis menetapkan *Unique Identifier* (UUID) yang menjadi kunci privasi untuk semua transaksi pengguna tersebut.
- **Session Management:** *Token-based authentication* (JWT) yang secara otomatis mengelola sesi dan melindungi rute di frontend.

B. Manajemen Transaksi

- **Pencatatan Cepat:** Formulir *input* yang intuitif untuk memasukkan nominal, tanggal, deskripsi, dan tipe transaksi (pemasukan/pengeluaran).
- **Kategori Kustom:** Setiap tenant dapat membuat kategori pengeluaran dan pemasukan mereka sendiri yang sepenuhnya terisolasi dari tenant lain.
- **Riwayat Transaksi:** Tabel daftar transaksi dengan fitur filter sederhana, diurutkan berdasarkan tanggal terbaru.

C. Ringkasan Dashboard Finansial

- **Arus Kas & Saldo:** Kalkulasi otomatis real-time (Total Pemasukan - Total Pengeluaran) untuk siklus bulan berjalan.
- **Grafik Distribusi:** Visualisasi persentase pengeluaran berdasarkan kategori untuk analisis pola konsumsi yang cepat.
- **Indikator Limit Anggaran:** Sistem peringatan visual sederhana jika pengeluaran mendekati atau melampaui plafon yang ditetapkan pengguna.

3. Multi-Tenant User Flow (Deskriptif)

1. **Onboarding (Landing Page):** Pengguna tiba di halaman utama dan mengklik CTA "Mulai Gratis".
2. **Registrasi Identity:** Pengguna memasukkan data diri. Layanan Auth mendaftarkan akun dan menerbitkan JWT yang berisi `UUID` unik tenant.
3. **Inisialisasi Workspace:** Pengguna diarahkan ke Dashboard kosong. Aplikasi menyimpan JWT dengan aman dan menggunakannya untuk setiap request.
4. **First Data Entry:** Pengguna membuat kategori perdana (mis. "Makan") dan mencatat pengeluaran. Frontend menyisipkan `user_id` saat pengiriman data ke database.

- 5. **Validasi RLS:** PostgreSQL menerima query. Supabase RLS mengevaluasi JWT secara native. Jika `auth.uid()` cocok dengan baris yang dituju, transaksi disimpan.
- 6. **Konsumsi Data:** Pengguna melihat grafik. Semua query ``SELECT`` secara otomatis terfilter oleh RLS di level database, sehingga mustahil memuat data milik tenant lain.
- 7. **Session End:** Pengguna keluar (logout). Sesi JWT dihapus bersih, memitigasi risiko keamanan akses di perangkat.

4. Database Schema (PostgreSQL)

Desain database relasional ini dioptimalkan untuk arsitektur multi-tenant, di mana kolom `user_id` menjadi pivot krusial pada tabel-tabel transaksional.

Table: profiles

Terhubung otomatis dengan `auth.users`.

Kolom	Tipe Data	Keterangan
id	uuid	Primary Key. Merujuk ke <code>auth.users.id</code> .
full_name	text	Nama tampilan pengguna.
created_at	timestampz	Tanggal inisiasi tenant.

Table: categories

Katalog kategori yang bersifat unik per pengguna.

Kolom	Tipe Data	Keterangan
id	uuid	Primary Key, auto-generated.
user_id	uuid	Foreign Key -> <code>profiles.id</code> . Wajib untuk RLS.
name	text	Nama kategori (misal: Transportasi).
type	text	'income' atau 'expense'.

Table: transactions

Tabel fakta utama tempat perekaman data finansial.

Kolom	Tipe Data	Keterangan
id	uuid	Primary Key.
user_id	uuid	Foreign Key -> profiles.id. Wajib untuk RLS.
category_id	uuid	Foreign Key -> categories.id.
amount	numeric	Nominal transaksi (mendukung desimal/presisi tinggi).
transaction_date	timestampz	Waktu terjadinya transaksi.
description	text	Catatan tambahan (opsional).

5. Security & Row Level Security (RLS) Blueprint

Lapisan keamanan paling kritis berada di level database. Dengan mengaktifkan RLS, kita menjamin secara matematis bahwa tidak ada **query** yang bisa menembus batas antar-tenant, terlepas dari kelemahan apa pun yang mungkin ada di level API atau Frontend.

"Pengguna hanya diizinkan untuk melihat (SELECT), membuat (INSERT), mengubah (UPDATE), dan menghapus (DELETE) baris data di mana nilai kolom `user\_id` sama persis dengan identifier `auth.uid()` yang terekstrak dari token JWT yang sedang aktif."

Implementasi SQL Policy pada Supabase:

```
-- Mengaktifkan RLS pada tabel sensitif
ALTER TABLE transactions ENABLE ROW LEVEL SECURITY;
ALTER TABLE categories ENABLE ROW LEVEL SECURITY;

-- Policy untuk menjamin isolasi Tenant 100% pada tabel transactions
CREATE POLICY "Strict Tenant Isolation for Transactions" ON transactions
FOR ALL
USING (auth.uid() = user_id)
WITH CHECK (auth.uid() = user_id);
```

6. Cloud Engineering: Setup Proyek & Arsitektur

Bagian ini diuraikan oleh Cloud Engineer untuk memulai pengembangan frontend menggunakan **Vite + React.js** yang terintegrasi dengan Supabase.

1. Struktur Folder (Component-Based & Modular)

```
budgeting-saas/
├── public/                # Aset statis & favicon
├── src/
│   ├── assets/           # Gambar, ilustrasi, font
│   ├── components/       # Reusable UI components
│   │   ├── layout/       # Navbar, Sidebar, Page Wrapper
│   │   └── ui/           # Buttons, Inputs, Cards (mis: Shadcn UI)
│   ├── features/         # Domain-specific components
│   │   ├── auth/        # Formulir Login, Register
│   │   ├── dashboard/   # Chart, Summary Cards
│   │   └── transactions/ # Tabel Riwayat, Form Entry
│   ├── lib/              # Eksternal API clients
│   │   └── supabase.js   # Supabase client init
│   ├── pages/            # Komponen halaman (Home, Dashboard)
│   ├── routes/           # Setup React Router & Protected Routes
│   ├── App.jsx           # Root component
│   └── main.jsx          # Entry point React
├── .env.local            # Secrets & Keys (JANGAN di-commit)
├── package.json
└── vite.config.js
```

2. Skrip Inisialisasi Terminal (Vite Setup)

```
# Inisialisasi kerangka proyek dengan Vite + React
npm create vite@latest budgeting-saas -- --template react

# Masuk ke folder kerja
cd budgeting-saas

# Install dependensi standar
npm install

# Install pustaka spesifik (Supabase, Router, Icons)
npm install @supabase/supabase-js react-router-dom lucide-react

# Jalankan server pengembangan lokal
npm run dev
```

3. Panduan Integrasi React - Supabase Client

Langkah A: Konfigurasi Environment Variables

Buat file `.env.local` di root proyek dan isikan kredensial Supabase:

```
VITE_SUPABASE_URL=https://<your-project-id>.supabase.co
VITE_SUPABASE_ANON_KEY=<your-anon-key>
```

Langkah B: Setup Client (`src/lib/supabase.js`)

Buat jembatan penghubung yang akan dipakai di seluruh aplikasi:

```
import { createClient } from '@supabase/supabase-js';

const supabaseUrl = import.meta.env.VITE_SUPABASE_URL;
const supabaseAnonKey = import.meta.env.VITE_SUPABASE_ANON_KEY;

export const supabase = createClient(supabaseUrl, supabaseAnonKey);
```

Langkah C: Verifikasi Koneksi & Session Management (`src/App.jsx`)

```

import { useEffect, useState } from 'react';
import { supabase } from '../lib/supabase';

function App() {
  const [session, setSession] = useState(null);

  useEffect(() => {
    // Cek sesi aktif saat mounting
    supabase.auth.getSession().then(({ data: { session } }) => {
      setSession(session);
    });

    // Listener otomatis untuk state login/logout
    const { data: { subscription } } = supabase.auth.onAuthStateChange(
      (_event, session) => {
        setSession(session);
      }
    );

    return () => subscription.unsubscribe();
  }, []);

  return (
    <div style={{ padding: '2rem', fontFamily: 'sans-serif' }}>
      <h1>MVP SaaS Personal Budgeting</h1>
      {session ? (
        <div style={{ padding: '1rem', background: '#d1fae5' }}>
          <p>Status: <strong>Terautentikasi</strong></p>
          <p>Tenant ID Aktif: {session.user.id}</p>
        </div>
      ) : (
        <p>Menunggu sesi aktif. Silakan implementasikan form login.</p>
      )}
    </div>
  );
}

export default App;

```